Gen AI Hands-On 1

Name: B Neeharika **SRN: PES2UG23CS114**

Sem/Sec: 6B **Date: 27/01/2026**

Observation Notes:

1. Seed Value Experiment

A **seed value** initializes the random number generator. In NLP generation models, randomness is used during sampling (next-token selection). Setting a seed ensures reproducibility.

```
[4]
✓ 0s
set_seed(42)
```

When the same seed value is used multiple times (e.g., `set_seed(42)`):
The generated output remains exactly the same every time, same word sequence, same sentence structure, same probability path in token selection
When the seed value is changed (e.g., `set_seed(10)` → `set_seed(99)`):
Output changes, sentence structure changes, word choices change, meaning may slightly drift, creativity varies
A seed value initializes the randomness used during text generation. When the same seed is used, the model follows the same probability path and produces identical output every time. Changing the seed changes the randomness path, leading to different word choices and sentence structures. This shows how seeds control reproducibility versus diversity in generated text.

2. Prompt Change Effect

The prompt acts as the starting context for the model. Even a small change in the prompt alters the probability distribution of the next tokens, which can completely change the direction, meaning, and focus of the generated text. This highlights how language models are highly context-driven.

Step 2: Define a Prompt

Both models will complete this sentence.

```
[5]
✓ 0s
prompt = "Generative AI is a revolutionary technology that"
```

Step 3: Fast Model (`distilgpt2`)

Let's see how the smaller model performs.

[6] 31s

```python
# Initialize the pipeline with the specific model
fast_generator = pipeline('text-generation', model='distilgpt2')

# Generate text
output_fast = fast_generator(prompt, max_length=50, num_return_sequences=1)
print(output_fast[0]['generated_text'])
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
config.json: 100%  762/762 [00:00<00:00, 21.2kB/s]
model.safetensors: 100%  353M/353M [00:06<00:00, 75.8MB/s]
generation_config.json: 100%  124/124 [00:00<00:00, 3.41kB/s]
tokenizer_config.json: 100%  26.0/26.0 [00:00<00:00, 830B/s]
vocab.json: 100%  1.04M/1.04M [00:00<00:00, 5.87MB/s]
merges.txt: 100%  456k/456k [00:00<00:00, 2.64MB/s]
tokenizer.json: 100%  1.36M/1.36M [00:00<00:00, 6.96MB/s]
Device set to use cpu
Truncation was not explicitly activated but `max_length` is provided a specific value, please use `truncation=True` to explicitly truncate examples to max length. Defaulting to 'longest_fir
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
Both `max_new_tokens` (=256) and `max_length`(=50) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface
Generative AI is a revolutionary technology that is designed to work with existing AI systems. It has been developed by the University of California, Berkeley. Its research team is the lead


The research team led by Professor Daniel Kranz, from the University of California, Berkeley, has developed a program to learn how to use the AI to improve the performance of the software.
The research team developed the program to learn how to use the AI to improve the performance of the software. It has been developed by the University of California, Berkeley, Berkeley, and
The research team developed the program to learn how to use the AI to improve the performance of the software. It has been developed by
```

Step 4: Standard Model (`gpt2`)

Now let's try the standard model.

[7] 27s

```python
smart_generator = pipeline('text-generation', model='gpt2')

output_smart = smart_generator(prompt, max_length=50, num_return_sequences=1)
print(output_smart[0]['generated_text'])
```

```
config.json: 100%  665/665 [00:00<00:00, 30.1kB/s]
model.safetensors: 100%  548M/548M [00:07<00:00, 114MB/s]
generation_config.json: 100%  124/124 [00:00<00:00, 9.06kB/s]
tokenizer_config.json: 100%  26.0/26.0 [00:00<00:00, 2.19kB/s]
vocab.json: 100%  1.04M/1.04M [00:00<00:00, 20.5MB/s]
merges.txt: 100%  456k/456k [00:00<00:00, 4.14MB/s]
tokenizer.json: 100%  1.36M/1.36M [00:00<00:00, 29.1MB/s]
Device set to use cpu
Truncation was not explicitly activated but `max_length` is provided a specific value, please use `truncation=True` to explicitly truncate
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
Both `max_new_tokens` (=256) and `max_length`(=50) seem to have been set. `max_new_tokens` will take precedence. Please refer to the docume
Generative AI is a revolutionary technology that allows users to build AI that can help solve complex problems. It brings together hundreds

The AI is a model of human intelligence, and has many aspects that are similar to artificial intelligence. It can learn from humans, and it

It is the main driving force behind the new Artificial Intelligence, and the AI is very important to the success of AI. The new AI is desig

The AI is designed to be scalable and adaptable to different environments. It can be used to solve complex problems without relying on huma

The new AI is designed to work out problems that need to be solved in a way that is
```

Analysis: Compare the two outputs. Does the standard model stay more on topic? Does the fast model drift into nonsense?

3. POS tagging

After running the POS tagging code, a LookupError occurred because the required NLTK tokenizer resource punkt_tab was not available in the environment. Downloading the necessary NLTK resources (punkt, punkt_tab, and averaged_perceptron_tagger) resolved the error. Once the resources were installed, the sentence was successfully tokenized and POS tags were generated correctly.

4. Hugging Face Platform

Hugging Face is an open-source ecosystem that provides pretrained models, tokenizers, datasets, and the Transformers library. It simplifies model usage through high-level APIs like pipelines, allowing developers to easily perform tasks such as text generation, summarization, and question answering.

5. Differences between Plain Model and Distilled Model

Plain Model	**Distilled Model**
A plain model is larger and provides better accuracy due to more parameters.	A distilled model is smaller and faster but slightly less accurate.
Plain models require more memory and computation power.	Distilled models consume less memory and run efficiently on CPUs.

6. Differences between BERT vs GPT

BERT	**GPT**
BERT is an encoder-only model that understands text using bidirectional context.	GPT is a decoder-only model that generates text one word at a time from left to right.
BERT is mainly used for understanding tasks like QA, NER, and classification.	GPT is mainly used for text generation tasks like chatbots and story writing.
BERT can fill in missing words because it sees the full sentence context.	GPT cannot fill blanks directly because it predicts only the next word.

7. Differences between BERT vs BART

BERT	BART
BERT focuses on understanding text and does not generate new sentences.	BART can both understand and generate text because it uses encoder and decoder.
BERT is suitable for classification and extractive QA.	BART is suitable for summarization and translation tasks.

8. Differences between Encoder vs Decoder

Encoder	Decoder
Encoder reads and understands the input text by learning contextual representations.	Decoder generates output text by predicting the next token sequentially.
Encoder is used for understanding tasks like QA and classification.	Decoder is used for generation tasks like text completion and chat.

9. Accuracy Loss in Distillation

During distillation, the student model learns a compressed representation of the teacher model. This compression removes some subtle patterns and fine-grained knowledge, which leads to a small reduction in accuracy but improves speed and efficiency.

10. Latency

Latency refers to the time taken by a model to generate a response. It depends on factors such as model size, hardware used, input length, and whether the model is distilled. Smaller and distilled models generally have lower latency.

11. LLM as CPU, Context as RAM

The language model acts like a CPU that processes information, while the prompt or context acts like RAM that holds temporary working data. Larger contexts consume more memory and influence how the model reasons and generates output.

12. Transformers Library

The Transformers library by Hugging Face provides ready-to-use implementations of transformer architectures along with pretrained weights and training utilities. It supports NLP, vision, and multimodal tasks, making it a core tool in modern AI development.

13. Transformer Architecture

A transformer consists of an encoder and a decoder. The encoder reads and understands input text by creating meaningful representations, while the decoder generates output text by predicting tokens one at a time based on context.

14. GPT-2 Architecture

GPT-2 is a decoder-only transformer model. It uses self-attention and causal masking to ensure tokens are generated in sequence, making it suitable for autoregressive text generation tasks.

15. Next Word Prediction

Text generation involves tokenizing the input, converting tokens into embeddings, processing them through transformer layers with self-attention, applying a softmax function to get probabilities, and sampling the next token based on those probabilities.

16. Softmax and Top-p Sampling

Softmax converts model outputs into probability distributions over the vocabulary. Top-p sampling selects tokens from the smallest set whose cumulative probability exceeds a threshold, balancing coherence and randomness in generated text.

17. Tokenization

Tokenization converts raw text into numerical tokens that the model can process. Techniques like subword tokenization and byte-pair encoding help models handle unknown words and reduce vocabulary size.

18. Self-Attention

Self-attention allows each word in a sentence to focus on other words to understand context. This helps the model distinguish meanings of the same word used in different situations by analyzing surrounding tokens.

19. NER

When an NER model reads a sentence, it doesn't just see words; it assigns them labels.